LIGHTYEAR

MS85 iDempiere schema alignment and native business journey

Evidence recorded 26 September 2026, following MS84.

Status: bounded native journey equivalence passed. Full schema equivalence, full application equivalence and platform qualification remain unclaimed.

Customer benefit

MS84 showed that corrected migration histories could reach matching admitted data states. MS85 asks the next practical question: can the real application perform useful business work on those databases and produce matching results?

It first repairs observed schema disagreements in isolated copies, then runs the same iDempiere business logic on Oracle and PostgreSQL. A customer buys two units, receives a shipment and invoice, and pays. Both databases finish with the same verified business outcome, including inventory and accounting.

This supplies an inspectable example of application compatibility. It is one end-to-end scenario across six stages, not comprehensive certification of six application areas.

Schema first

The starting data comes from the admitted MS84 historical replay. Original databases were preserved; changes were applied only to isolated successors.

Check or repair	Result
Common base tables	913
Foreign keys added	36: 32 Oracle and 4 PostgreSQL
Matching foreign-key relationships after repair	3,826
Nullability conflicts repaired	30
Common columns agreeing on nullability	18,852
Matching primary-key column sets	900
Matching unique-key column sets	917
Table-row multisets changed by schema repairs	0
Native constraint probes	30 passed per engine

The user-selected shared application dictionary resolves the 30 nullability conflicts: 23 Oracle columns become optional, five Oracle columns become required, and two PostgreSQL columns become required. Both databases' dictionary rows agree; native preflight checks establish that tightening constraints will not reject existing data. Foreign-key comparisons include ordered columns, referential actions, validation, deferral and enforcement state.

The 30 rollback-only probes on each engine cover a positive update and primary key, foreign key, required-column and Y/N rejection checks on each of six journey tables. Deferred constraints are forced immediate, expected native error classes are checked, and affected rows are read back after rollback.

Twenty separate paired expression probes produce 11 matching values, two matching overflow rejections and seven differences, with none inconclusive. The differences concern numeric precision, empty strings, multibyte CHAR, fractional time, UUIDs and JSON. A separate clock observation also differs. These are standalone expressions, not proof of what constrained application tables accept. For example, Oracle JSON columns have additional `IS JSON` checks. Raw findings remain in the schema assessment.

Then the real application

The identical Java harness runs through iDempiere's model and document-workflow APIs, using source commit `731515dcdd5278b843db33b9d3109d155b881951`. Business writes are performed by the application. Independent native readback checks committed records after the application exits.

The engines are **Oracle 26ai Free (23.26.3.0.0)** and **PostgreSQL 15.19** in local Docker. The reference client is GardenWorld. No GCP resources were started.

Stage or outcome	Verified on each engine
Customer	New customer and address saved and linked
Product	New stocked product, opening inventory 10, unit price 10
Order	Completed, quantity 2, net total 20
Shipment	Completed, quantity delivered 2
Invoice	Completed, total 20, ultimately marked paid
Payment	Completed and allocated, amount 20
Closing inventory	8 units; two inventory movement records
Accounting	12 entries checked; six balanced document/schema/currency groups
New application issues	0

The final runs use fresh verified starting copies and normal commit boundaries. Earlier setup attempts remain visible: one omitted a price-list entry; another held the scenario in one transaction and caused customer-visibility issues. The archive retains four diagnostic executions and two successful final executions. The diagnostics are not counted as successful journeys.

Differences are validated, not discarded

Final readback compares 913 common base tables. It retains **305 new raw field differences**: 300 admitted by explicit application rules and five by existing UUID rules. **Zero differences remain unexplained.** The 5,540 unchanged differences already admitted by MS84 are accounted for separately.

New UUIDs must be valid, unique and absent from the starting data. Generated times must fit native execution windows; existing creation times cannot be rewritten. Workflow durations are bounded, and workflow text can vary only in exact decimal rendering. Runtime database identity must match the observed isolated target and pinned driver's behavior. Both raw values and the admitting rule remain available for every accepted difference.

New application issues, unknown differences, missing or extra row keys, and writes outside the declared journey footprint block admission. A successful test-process exit alone is insufficient.

Claim boundaries

The application receipt states:

```
status: passed-bounded-journey-equivalence
bounded_journey_equivalence: true
application_equivalence: false
schema_equivalence: false
platform_qualification: false
independently_attested: false
```

The true flag covers this priced, stocked customer-to-payment scenario on the two recorded engines. The false flags cover broader claims that this scenario cannot establish. Datatypes, defaults, broader constraint enforcement, indexes, views, routines, triggers, privileges, policies and sequences retain open obligations. Catalog declaration differences are not automatically defects.

There is no claim here about all tax rules, valuation methods, reversals, concurrency, performance, browser behavior or production operations. Oracle 19c, AlloyDB and the separate Oracle behavior-catalog conformance gate are outside this execution. Evidence is content-hash-bound native observation, not independent attestation or a third-party signature.

Evidence and validation

- Schema report and reproduction instructions, assessment, receipt and native evidence.
- Application report and reproduction instructions, receipt, comparison, exact Java harness and native evidence.
- Cleanup observation: all four experiment database containers stopped, with data retained.

Five Python modules add catalog assessment, expression probes, rollback-only constraint probes, explicit application difference rules and independent business-outcome verification. Seven test modules cover those contracts and replay the published evidence offline. The affected calibration suite runs 175 tests: **173 passed, two skipped** locally. CI includes the same modules on Linux and Windows. Publication checks reconstruct sealed states, check archive hashes and retained rows, and recompute the admitted business outcomes.

Credentials, private runtime settings and database images are excluded from the publication. Offline verification needs no running database.

What this enables next

The same evidence pattern can now be extended to additional business scenarios: tax, discounts, partial shipment, credit notes and reversals, then concurrent activity and failure recovery. Each extension should declare its starting state, expected outcomes and permitted raw differences before execution. Broader schema behavior and operational qualification remain separate work.